

algebra. Let's consider the following code, which is used instead of equations:

```
w=xld.open_workbook("filename.xls").sheet_by_name("Sheet1")
x,y=w.col_values(0),w.col_values(1)
degree=1
weight=1/array(x)**2
f=zeros(w.nrows,degree+1)
for i in range(0,w.nrows):
    for j in range(0,degree+1):
        f[i,j]=x[i]**j
q=diag(weight)
ft=dot(transpose(f),q)
ftf=dot(ft,f)
# fTf=ftf+diag([0.001]*(degree+1))
fTy=dot(ft,y)
coef=dot(inv(ftf),fTy)
xx=linspace(min(x),max(x),1000)
yp=polyval(coef[:-1],xx)
```

In the example, the degree is equal to 1 (a linear model) and the weight is equal to $1/x^2$ (this is a weighted model). For a simple linear model, degree=1 and weight=1 (or $weight=1/array(x)**0$). For a simple quadratic model, degree=2 and weight=1. With some simple matrix calculations, it's possible to know the coefficients *coef* of the model and then plot the curve *xx* vs *yp*. For fitting through zero, set $weight=1/array(x)**0$ and for *j* in *range(1,degree+1)*; add a small constant, for example 0.001, to the diagonal of *fTf*. For a linear model, the predicted values (x values back calculated) are calculated with $predicted=(y-intercept)/slope$ and, for a quadratic model, with $predicted=(-b+sqrt(b**2-(4*a*delta)))/(2*a)$ in which $delta=c-y$. The *a*, *b* and *c* values are about the coefficient of the second degree term, the coefficient of the first degree term and the constant value, respectively. The accuracy is then calculated as $accuracy=predicted*100/x$. The following tables present a linear simple and a linear weighted model for

the same experimental data. I'm not a big fan of the squared correlation coefficient, but, considering only that, I should choose the linear model simple, because it's the one with the highest value of R^2 . I think it would be better to consider the 'Accuracy' column in both tables and choose the weighted model. In this particular example, we must also consider what is already mentioned in the 'Introduction' section about the values established by law. Carry out these calculations using a spreadsheet (and without using macros). As I think that's unnecessarily complicated.

Linear simple

```
Intercept -9343.97333
Slope 16984.40793
R2 0.99860
```

X	Y	Predicted	Accuracy
0.1	1633	0.65	646.30
0.5	7610	1.00	199.64
1.0	14930	1.43	142.92
2.5	35347	2.63	105.25
5.0	69403	4.64	92.73
10.0	141643	8.89	88.90
25.0	402754	24.26	97.05
50.0	850161	50.61	101.21

Linear weighted 1/x2

```
Intercept 123.19563
Slope 15010.23851
R2 0.99422
```

X	Y	Predicted	Accuracy
0.1	1633	0.10	100.58
0.5	7610	0.50	99.76
1.0	14930	0.99	98.64
2.5	35347	2.35	93.87
5.0	69403	4.62	92.31
10.0	141643	9.43	94.28
25.0	402754	26.82	107.29
50.0	850161	56.63	113.26

Figure 1 shows the plot for another data set, a simulation more or less typical for a pharmacokinetics study.

Another way is to build a certain matrix via *vstack* and then apply *lstsq* on it. Each *A* matrix created with *vstack* has the structure shown in the following examples. There are two important things: if the model is built to fit through zero, there is a column of zeros and, if the model is quadratic, there is a column of squared *x*. The coefficients of each model are *a*, *b* and *c* (if quadratic).

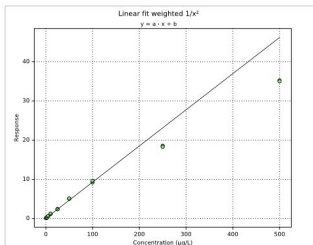


Figure 1: A linear weighted model